OpenGL & GLUT

Complete Study Notes

Tutorial 01 — Primitives, Color & 2D Drawing

C++ | OpenGL | GLUT | Computer Graphics

About These Notes

This document covers all OpenGL/GLUT tutorial examples from the first session. Each section includes complete C++ source code, pixel coordinate calculations, and step-by-step explanations of how the rendering works.

Table of Contents

Concept 01

OpenGL Coordinate System

OpenGL does **not** use pixel coordinates directly. The entire window is mapped to a **-1 to +1** range on both X and Y axes, regardless of actual window size.

Coordinate Positions (320×320 window)

Position	OpenGL X	OpenGL Y	Pixel X	Pixel Y
Center	0.0	0.0	160	160
Right edge	1.0	0.0	320	160
Left edge	-1.0	0.0	0	160
Top edge	0.0	1.0	160	0
Bottom edge	0.0	-1.0	160	320
Top-right corner	1.0	1.0	320	0
Bottom-left	-1.0	-1.0	0	320

Pixel Conversion Formula

```
pixel_x = (x + 1) x (window_width  / 2)
pixel_y = (1 - y) x (window_height / 2)

// For a 320x320 window (half = 160):
pixel_x = (x + 1) x 160
pixel_y = (1 - y) x 160
```

✓ Y is **flipped** — In OpenGL +Y goes UP, but screen pixels count from 0 at the TOP. That is why pixel_y uses (1 - y), not (1 + y).

Example 01

Draw a White Window

The simplest OpenGL program — opens a 320x320 white window.

```
#include <windows.h>   // MS Windows support
#include <GL/glut.h>   // GLUT library

void display() {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); // White (R=1, G=1, B=1, A=1)
    glClear(GL_COLOR_BUFFER_BIT);           // Apply the background color
    glFlush();                              // Send commands to GPU
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);                  // Initialize GLUT
    glutCreateWindow("OpenGL Setup Test");  // Window title bar text
    glutInitWindowSize(320, 320);           // Window width x height
    glutDisplayFunc(display);               // Register paint callback
    glutMainLoop();                         // Start the event loop
    return 0;
}
```

Key Functions

Function	What it does
`glutInit()`	Initializes the GLUT library
`glutCreateWindow(title)`	Creates the OS window with a title bar
`glutInitWindowSize(w, h)`	Sets window size in pixels
`glutDisplayFunc(fn)`	Registers the function that paints the window
`glutMainLoop()`	Starts the infinite event loop
`glClearColor(R, G, B, A)`	Sets background fill color (values 0.0 to 1.0)
`glClear(GL_COLOR_BUFFER_BIT)`	Fills screen with the clear color
`glFlush()`	Forces all pending GL commands to execute

Example 02

Draw a Point

Draws a single red dot at the exact center of a black window.

```
void display() {
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f); // Black background
    glClear(GL_COLOR_BUFFER_BIT);

    glPointSize(5.0);                     // Point diameter = 5 pixels

    glBegin(GL_POINTS);                   // Start drawing points
        glColor3f(1.0f, 0.0f, 0.0f);     // Red color (R=1, G=0, B=0)
        glVertex2f(0.0f, 0.0f);           // Place point at center (0, 0)
    glEnd();

    glFlush();
}
```

Pixel Calculation — 320x320 window

Coordinate	Formula	Result
x = 0.0	pixel_x = (0.0 + 1) x 160	= 160 (center horizontally)
y = 0.0	pixel_y = (1 - 0.0) x 160	= 160 (center vertically)

■ The point (0.0, 0.0) maps to pixel (160, 160) — exactly the center of the 320x320 window.

Example 03

Draw a Line

Draws a thick red horizontal line from center (0,0) to the right edge (1,0).

```
void display() {
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glLineWidth(7.5);                     // Line thickness in pixels

    glBegin(GL_LINES);    // GL_LINES needs vertex PAIRS
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex2f(0.0f, 0.0f);  // Start: center (0, 0)
        glVertex2f(1.0f, 0.0f);  // End:   right edge (1, 0)
    glEnd();
    glFlush();
}
```

Pixel Calculation — 320x320 window

Vertex	OpenGL (x, y)	pixel_x = (x+1)x160	pixel_y = (1-y)x160
Start	(0.0, 0.0)	(0+1)x160 = 160	(1-0)x160 = 160
End	(1.0, 0.0)	(1+1)x160 = 320	(1-0)x160 = 160

✓ GL_LINES always reads vertices in **pairs**. First pair = first line, second pair = second line, and so on.

Example 04

Draw X & Y Axis

Draws 4 lines from center (0,0) in all 4 directions to form a full axis cross.

```
void display() {
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(0.5);

    glBegin(GL_LINES);
        glColor3f(1.0f, 0.0f, 0.0f);

        // +X axis: center -> right
        glVertex2f( 0.0f,  0.0f);  glVertex2f( 1.0f,  0.0f);

        // +Y axis: center -> top
        glVertex2f( 0.0f,  0.0f);  glVertex2f( 0.0f,  1.0f);

        // -X axis: center -> left
        glVertex2f( 0.0f,  0.0f);  glVertex2f(-1.0f,  0.0f);

        // -Y axis: center -> bottom
        glVertex2f( 0.0f,  0.0f);  glVertex2f( 0.0f, -1.0f);
    glEnd();
    glFlush();
}
```

Line	From	To	Direction
+X axis	(0, 0)	(1, 0)	Right ->
+Y axis	(0, 0)	(0, 1)	Up ^
-X axis	(0, 0)	(-1, 0)	Left <-
-Y axis	(0, 0)	(0, -1)	Down v

Example 05

Draw a Polygon

Draws a filled yellow hexagon in the first quadrant (top-right area) using 6 vertices.

```
void initGL() {
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_POLYGON);              // Filled & auto-closed shape
        glColor3f(1.0f, 1.0f, 0.0f); // Yellow (R=1, G=1, B=0)
        glVertex2f(0.4f,  0.2f);      // P1 - bottom-left
        glVertex2f(0.6f,  0.2f);      // P2 - bottom-right
        glVertex2f(0.7f,  0.4f);      // P3 - mid-right
        glVertex2f(0.6f,  0.6f);      // P4 - top-right
        glVertex2f(0.4f,  0.6f);      // P5 - top-left
        glVertex2f(0.3f,  0.4f);      // P6 - mid-left
    glEnd();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutCreateWindow("Vertex, Primitive & Color");
    glutInitWindowSize(320, 320);
    glutInitWindowPosition(50, 50);   // Set window position on screen
    glutDisplayFunc(display);
    initGL();
    glutMainLoop();
    return 0;
}
```

All 6 Points — Pixel Calculation (320x320 window)

Point	OpenGL (x,y)	pixel_x=(x+1)x160	pixel_y=(1-y)x160	Position
P1	(0.4, 0.2)	(1.4)x160 = 224	(0.8)x160 = 128	Bottom-left
P2	(0.6, 0.2)	(1.6)x160 = 256	(0.8)x160 = 128	Bottom-right
P3	(0.7, 0.4)	(1.7)x160 = 272	(0.6)x160 = 96	Mid-right
P4	(0.6, 0.6)	(1.6)x160 = 256	(0.4)x160 = 64	Top-right
P5	(0.4, 0.6)	(1.4)x160 = 224	(0.4)x160 = 64	Top-left
P6	(0.3, 0.4)	(1.3)x160 = 208	(0.6)x160 = 96	Mid-left

✓ GL_POLYGON automatically connects the last vertex back to the first — no need to repeat the starting point at the end.

Example 06

Four Objects in Four Quadrants

Draws 4 different shapes in 4 different quadrants using different colors and primitives.

```
void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Q1 (top-right): Yellow Hexagon
    glBegin(GL_POLYGON);
        glColor3f(1.0f, 1.0f, 0.0f);
        glVertex2f(0.4f,0.2f); glVertex2f(0.6f,0.2f); glVertex2f(0.7f,0.4f);
        glVertex2f(0.6f,0.6f); glVertex2f(0.4f,0.6f); glVertex2f(0.3f,0.4f);
    glEnd();

    // Q2 (top-left): Red Triangle
    glBegin(GL_TRIANGLES);
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex2f(-0.9f, 0.3f);
        glVertex2f(-0.5f, 0.3f);
        glVertex2f(-0.7f, 0.6f);
    glEnd();

    // Q3 (bottom-left): Green Square
    glBegin(GL_QUADS);
        glColor3f(0.0f, 1.0f, 0.0f);
        glVertex2f(-0.8f,-0.8f); glVertex2f(-0.5f,-0.8f);
        glVertex2f(-0.5f,-0.5f); glVertex2f(-0.8f,-0.5f);
    glEnd();

    // Q4 (bottom-right): Orange Triangle
    glBegin(GL_TRIANGLES);
        glColor3ub(232, 133, 20);   // RGB 0-255 range
        glVertex2f( 0.5f,-0.8f);
        glVertex2f( 0.7f,-0.8f);
        glVertex2f( 0.6f,-0.4f);
    glEnd();

    glFlush();
}
```

Shape	Primitive	Color	Quadrant	Vertices
Hexagon	GL_POLYGON	Yellow	Q1 (+X, +Y)	6
Triangle	GL_TRIANGLES	Red	Q2 (-X, +Y)	3
Square	GL_QUADS	Green	Q3 (-X, -Y)	4
Triangle	GL_TRIANGLES	Orange	Q4 (+X, -Y)	3

■ glColor3f() uses 0.0-1.0 float range. glColor3ub() uses 0-255 integer range. Both produce the same result.

Assignment 01

Rainbow Flag

Draw 6 equal horizontal color stripes across the full window using GL_QUADS.

■ Window Y range = -1 to +1 = **2 units** total. 6 stripes: each stripe = 2 / 6 = **0.333 units** tall. X always spans -1.0 to +1.0 (full width).

Stripe Coordinates

Stripe	Color	Top Y	Bottom Y	Color Function
1	Red	+1.0	+0.667	`glColor3f(1.0, 0.0, 0.0)`
2	Orange	+0.667	+0.333	`glColor3f(1.0, 0.5, 0.0)`
3	Yellow	+0.333	0.0	`glColor3f(1.0, 1.0, 0.0)`
4	Green	0.0	-0.333	`glColor3f(0.0, 0.5, 0.0)`
5	Blue	-0.333	-0.667	`glColor3f(0.0, 0.0, 1.0)`
6	Purple	-0.667	-1.0	`glColor3ub(148, 0, 211)`

Code — Part 1 (includes + stripes 1-3)

```
#include <windows.h>
#include <GL/glut.h>

void initGL() { glClearColor(0,0,0,1); }

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Stripe 1: RED  (y: +1.0 to +0.667)
    glBegin(GL_QUADS);
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex2f(-1.0f, 0.667f); glVertex2f( 1.0f, 0.667f);
        glVertex2f( 1.0f, 1.0f);   glVertex2f(-1.0f, 1.0f);
    glEnd();

    // Stripe 2: ORANGE  (y: +0.667 to +0.333)
    glBegin(GL_QUADS);
        glColor3f(1.0f, 0.5f, 0.0f);
        glVertex2f(-1.0f, 0.333f); glVertex2f( 1.0f, 0.333f);
        glVertex2f( 1.0f, 0.667f); glVertex2f(-1.0f, 0.667f);
    glEnd();

    // Stripe 3: YELLOW  (y: +0.333 to 0.0)
    glBegin(GL_QUADS);
        glColor3f(1.0f, 1.0f, 0.0f);
        glVertex2f(-1.0f, 0.0f);   glVertex2f( 1.0f, 0.0f);
        glVertex2f( 1.0f, 0.333f); glVertex2f(-1.0f, 0.333f);
    glEnd();
```

Code — Part 2 (stripes 4-6 + main)

```
// Stripe 4: GREEN  (y: 0.0 to -0.333)
    glBegin(GL_QUADS);
        glColor3f(0.0f, 0.5f, 0.0f);
        glVertex2f(-1.0f,-0.333f); glVertex2f( 1.0f,-0.333f);
        glVertex2f( 1.0f, 0.0f);   glVertex2f(-1.0f, 0.0f);
    glEnd();

    // Stripe 5: BLUE  (y: -0.333 to -0.667)
    glBegin(GL_QUADS);
        glColor3f(0.0f, 0.0f, 1.0f);
        glVertex2f(-1.0f,-0.667f); glVertex2f( 1.0f,-0.667f);
        glVertex2f( 1.0f,-0.333f); glVertex2f(-1.0f,-0.333f);
    glEnd();

    // Stripe 6: PURPLE  (y: -0.667 to -1.0)
    glBegin(GL_QUADS);
        glColor3ub(148, 0, 211);
        glVertex2f(-1.0f,-1.0f);   glVertex2f( 1.0f,-1.0f);
        glVertex2f( 1.0f,-0.667f); glVertex2f(-1.0f,-0.667f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutCreateWindow("Rainbow Flag");
    glutInitWindowSize(320, 320);
    glutDisplayFunc(display);
    initGL();
    glutMainLoop();
    return 0;
}
```

Assignment 02

AIUB Text

Draw the letters A, I, U, B using GL_LINES with thick strokes (glLineWidth = 8) across a 480x320 window. Each letter height spans y = -0.70 to +0.70.

Letter Structure

Letter	Color	Strokes	Key X Range
A	Red	2 diagonals + crossbar at y=0.0	x: -0.90 to -0.50
I	Green	Top bar + vertical stem + bottom bar	Stem at x=-0.28
U	Blue	2 verticals + 3-part curved bottom	x: 0.08 to 0.42
B	Yellow	Spine + 3 horizontals + 2x3-segment bumps	x: 0.58 to 0.92

Code - Part 1: setup + letters A & I

```
#include <windows.h>
#include <GL/glut.h>

void initGL() { glClearColor(0,0,0,1); }

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(8.0f);

    // --- Letter A (Red) ---
    glBegin(GL_LINES);
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex2f(-0.90f,-0.70f); glVertex2f(-0.70f, 0.70f); // Left diagonal
        glVertex2f(-0.70f, 0.70f); glVertex2f(-0.50f,-0.70f); // Right diagonal
        glVertex2f(-0.80f, 0.00f); glVertex2f(-0.60f, 0.00f); // Crossbar
    glEnd();

    // --- Letter I (Green) ---
    glBegin(GL_LINES);
        glColor3f(0.0f, 1.0f, 0.0f);
        glVertex2f(-0.38f, 0.70f); glVertex2f(-0.18f, 0.70f); // Top bar
        glVertex2f(-0.28f, 0.70f); glVertex2f(-0.28f,-0.70f); // Stem
        glVertex2f(-0.38f,-0.70f); glVertex2f(-0.18f,-0.70f); // Bottom bar
    glEnd();
```

Code - Part 2: letters U & B + main()

```
// --- Letter U (Blue) ---
    glBegin(GL_LINES);
        glColor3f(0.0f, 0.5f, 1.0f);
        glVertex2f(0.08f, 0.70f); glVertex2f(0.08f,-0.50f); // Left vertical
        glVertex2f(0.08f,-0.50f); glVertex2f(0.18f,-0.70f); // BL curve
        glVertex2f(0.18f,-0.70f); glVertex2f(0.32f,-0.70f); // Bottom
        glVertex2f(0.32f,-0.70f); glVertex2f(0.42f,-0.50f); // BR curve
        glVertex2f(0.42f,-0.50f); glVertex2f(0.42f, 0.70f); // Right vertical
    glEnd();

    // --- Letter B (Yellow) ---
    glBegin(GL_LINES);
        glColor3f(1.0f, 1.0f, 0.0f);
        glVertex2f(0.58f, 0.70f); glVertex2f(0.58f,-0.70f); // Spine
        glVertex2f(0.58f, 0.70f); glVertex2f(0.78f, 0.70f); // Top
        glVertex2f(0.58f, 0.00f); glVertex2f(0.78f, 0.00f); // Middle
        glVertex2f(0.58f,-0.70f); glVertex2f(0.78f,-0.70f); // Bottom
        glVertex2f(0.78f, 0.70f); glVertex2f(0.90f, 0.50f); // Top bump
        glVertex2f(0.90f, 0.50f); glVertex2f(0.90f, 0.20f);
        glVertex2f(0.90f, 0.20f); glVertex2f(0.78f, 0.00f);
        glVertex2f(0.78f, 0.00f); glVertex2f(0.92f,-0.20f); // Bottom bump
        glVertex2f(0.92f,-0.20f); glVertex2f(0.92f,-0.50f);
        glVertex2f(0.92f,-0.50f); glVertex2f(0.78f,-0.70f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutCreateWindow("AIUB Text");
    glutInitWindowSize(480, 320);
    glutDisplayFunc(display);
    initGL();
    glutMainLoop();
    return 0;
}
```

Reference

Quick Reference Card

Drawing Primitives

Primitive	Vertices Needed	Description
GL_POINTS	1 per point	Dots drawn at each vertex position
GL_LINES	2 per line	Separate line segments — requires pairs
GL_TRIANGLES	3 per triangle	Filled triangles
GL_QUADS	4 per quad	Filled quadrilaterals / rectangles
GL_POLYGON	N vertices	Single filled polygon, auto-closed

Color Reference

Color	glColor3f(R, G, B)	glColor3ub(R, G, B)
Red	(1.0, 0.0, 0.0)	(255, 0, 0)
Green	(0.0, 1.0, 0.0)	(0, 255, 0)
Blue	(0.0, 0.0, 1.0)	(0, 0, 255)
Yellow	(1.0, 1.0, 0.0)	(255, 255, 0)
Orange	(1.0, 0.5, 0.0)	(255, 128, 0)
Purple	(0.5, 0.0, 0.5)	(148, 0, 211)
White	(1.0, 1.0, 1.0)	(255, 255, 255)
Black	(0.0, 0.0, 0.0)	(0, 0, 0)

Size & Width Functions

Function	Description	Example
`glPointSize(n)`	Point diameter in pixels	`glPointSize(5.0)`
`glLineWidth(n)`	Line thickness in pixels	`glLineWidth(8.0f)`
`glutInitWindowSize(w,h)`	Window dimensions	`glutInitWindowSize(320, 320)`
`glutInitWindowPosition(x,y)`	Window position on screen	`glutInitWindowPosition(50, 50)`

Coordinate Conversion Formula

```
// For any window size:
pixel_x = (x + 1) x (window_width  / 2)
pixel_y = (1 - y) x (window_height / 2)

// For a 320x320 window (half = 160):
pixel_x = (x + 1) x 160
pixel_y = (1 - y) x 160

// Quick examples:
// ( 0.0,  0.0) -> pixel (160, 160) -- center
// ( 1.0,  0.0) -> pixel (320, 160) -- right edge
// ( 0.0,  1.0) -> pixel (160,   0) -- top edge
// (-1.0, -1.0) -> pixel (  0, 320) -- bottom-left corner
```

Program Structure Template

```
#include <windows.h>
#include <GL/glut.h>

void initGL() {
    glClearColor(R, G, B, A);         // Background color
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);      // Clear screen

    // --- Draw your shapes here ---
    glLineWidth(n);                    // (optional) line width
    glPointSize(n);                    // (optional) point size

    glBegin(PRIMITIVE_TYPE);
        glColor3f(R, G, B);            // Set color
        glVertex2f(x, y);              // Define vertex
        // ... more vertices ...
    glEnd();

    glFlush();                         // Render to screen
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);             // Init GLUT
    glutCreateWindow("Window Title");  // Create window
    glutInitWindowSize(320, 320);      // Set size
    glutDisplayFunc(display);          // Set display callback
    initGL();                          // Your GL settings
    glutMainLoop();                    // Start event loop
    return 0;
}
```